INSTITUTO FEDERAL
Santa Catarina

Departamento Acadêmico de Eletrônica – **DAELN**
IFSC Câmpus Florianópolis

Programação C++

Expressões, variáveis e constantes

Prof. Matheus Leitzke Pinto
matheus.pinto@ifsc.edu.br

Expressões em C++

- Objetivos da aula:
  - Compreender as noções fundamentais de expressões e variáveis em programação.
  - Familiarizar-se com os operadores aritméticos, relacionais, lógicos e de atribuição em C++.
  - Aprender a declarar e inicializar variáveis em C++ e compreender os diferentes tipos de dados disponíveis.
  - Conhecer as regras de nomenclatura de variáveis e entender como escolher nomes apropriados.
  - Explorar como utilizar expressões e variáveis juntas para manipular e calcular dados.

Expressões em C++

- Ao final desta aula, os alunos devem estar confortáveis com a criação, manipulação e utilização de expressões e variáveis em programas C++, e devem estar preparados para continuar a explorar conceitos mais avançados na linguagem.

Expressões em C++

- Uma variável é um espaço de memória nomeado que pode armazenar um valor. As variáveis são usadas para representar dados que podem ser manipulados e usados em diferentes partes do programa.

- Uma expressão é uma combinação de valores, operadores e funções que, quando avaliada, produz um resultado. Elas são usadas para realizar cálculos e executar operações em programas C++

Expressões em C++

- O C++ oferece diversos operadores que podem ser usados em expressões, tais como:
  - Aritméticos: +, -, *, /, %
  - Relacionais: ==, !=, <, >, <=, >=
  - Lógicos: && (E), || (OU), ! (NÃO)
  - Atribuição: =, +=, -=, *=, /=, %=
  - Incremento/Decremento: ++, --
  - Bitwise: &, |, ~, ^

Expressões em C++

EXEMPLOS DE EXPRESSÕES

```
int soma = 5 + 3;
bool resultado = (a > b) && (c != 0);
```

Sumário de aula

- Tipos de dados básicos em C++
- Declaração e Inicialização de variáveis
- Constantes
- Overflow aritmético
- Expressões aritméticas
- Expressões relacionais
- Expressões lógicas
- Expressões compostas
- *Casting*

INSTITUTO FEDERAL
Santa Catarina

DAELN

Tipos de dados básicos em C++

Tipos de dados básicos em C++

INTEIROS

Tipo	Tamanho de armazenamento	Faixa de valores
char	1 byte	-128 à 127
unsigned char	1 byte	0 à 255
signed char	1 byte	-128 à 127
int	2 ou 4 bytes	-32,768 à 32,767 ou -2,147,483,648 à 2,147,483,647
unsigned int	2 ou 4 bytes	0 à 65,535 ou 0 à 4,294,967,295
short	2 bytes	-32,768 à 32,767
unsigned short	2 bytes	0 à 65,535
long	4 bytes	-2,147,483,648 à 2,147,483,647
unsigned long	4 bytes	0 à 4,294,967,295
long long	8 bytes	-9,22337204E-18 à 9,22337204E18
unsigned long long	8 bytes	0 à 1,84467441E19

Expressões em C++

EXEMPLOS DE DECLARAÇÕES DE NÚMEROS INTEIROS

```
int x = 0;

int caracol = -767888990;

unsigned int meuValor = 10;

long meuValor2 = 3;

char letra = 'a';

char numero = 64;
```

Tipos de dados em C

PONTOS FLUTUANTES

Tipo	Tamanho de armazenamento	Faixa de valores	Precisão
float	4 byte	1.2E-38 à 3.4E+38	6 casas decimais
double	8 byte	2.3E-308 à 1.7E+308	15 casas decimais
long double	10 byte	3.4E-4932 à 1.1E+4932	19 casas decimais

Expressões em C++

EXEMPLOS DE DECLARAÇÕES DE NÚMEROS EM PONTO FLUTUANTE

```
float pi = 3.14;

float pi_minus = -3.14;

float e = 2.71828;

double e_bigger = 2.718281828459045235360287;
```

Tipos de dados em C

BOOLEANOS

```
bool b;
...
b = true;
...
b = false;
```

☞ Booleanos em C++ irão ocupar um byte na memória

INSTITUTO FEDERAL
Santa Catarina

DAELN

Declaração e Inicialização de variáveis

Declaração e Inicialização de variáveis

- O nome de uma variável deve começar com uma letra ou underscore (_), seguido por letras, underscores ou dígitos.

- Letras maiúsculas e minúsculas são diferentes (case-sensitive).

- Não é permitido usar palavras-chave da linguagem como nomes de variáveis.

Declaração e Inicialização de variáveis

- Para usar uma variável em C++, é necessário declará-la e, opcionalmente, inicializá-la com um valor.

```
int idade; // Declaração

idade = 25; // Inicialização
```

Declaração e Inicialização de variáveis

- Também é possível fazer a declaração e inicialização em uma única linha:

```
float preco = 10.99;
```

Declaração e Inicialização de variáveis

- A declaração de uma variável pode ser feita em qualquer parte do código.
- A variável não pode ser utilizada antes da sua declaração.

```
int x;
x = 3; // correto
```

```
x = 1.044; // errado
float x;
```

Declaração e Inicialização de variáveis

- Variáveis de um mesmo tipo podem ser declaradas em uma mesma linha.

```
int x = 3, oi = 333, tchau = 477779;
```

Constantes

Constantes

- Constantes são os valores que não mudam em tempo de execução do programa.

- No C++ podemos ter:
  - **Constantes literais**;
  - **Constantes nomeadas**.

Constantes

- **Constantes literais** são todos aqueles números colocados no código.

- Exemplo:
  - 148, 2555, 65535 (decimal inteiro);
  - 3.5677, 457.008, 666.666 (decimal em ponto flutuante);
  - 0xFFC3, 0x3A, 0x173 (hexadecimal);
  - 027, 0567, 0666 (octal);
  - ‘A’, ‘D’, ‘3’, ‘Z’ (ASCII).

Constantes

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Constantes

- Em C++, inteiros literais são tratados como o tipo de dado int por padrão. Isso significa que se você escrever um número inteiro literal, como 42, ele será considerado um valor do tipo int.

- No entanto, você pode modificar o tipo de um literal usando sufixos ou prefixos específicos para indicar o tipo que você deseja. Isso permite que você utilize outros tipos de dados inteiros, como long, long long, unsigned int, etc.

Constantes

- Exemplos

```
// Usando 'L' para indicar long
long numeroLongo = 1567890123L;

// Usando 'u' para indicar unsigned int
unsigned int positivo = 100u;
```

Constantes

- Aqui estão alguns sufixos que você pode usar para modificar o tipo de um literal:

- L ou l: Indica long ou long long.
- U ou u: Indica unsigned int ou unsigned long.
- LL ou ll: Indica long long.
- ULL ou ull: Indica unsigned long long.
- F ou f: Indica float.
- LF ou lf: Indica long double.

Constantes

- **Constantes nomeadas** têm nomes e valores associados a eles e são usadas para tornar seu código mais legível e manutenível.

- Exemplo:

```
const int maxValue = 100;

constexpr double pi_number = 3.14159265359;
```

Constantes

- Também, podemos nomear uma constante com o uso da diretiva de pré-processamento `#define`.

- Exemplo:

```
#define MAX_VALUE 100

int main()
{
    // Isso será substituído por int x = 100 * 2
    // durante a compilação
    int x = MAX_VALUE * 2;
    return 0;
}
```

Constantes

- Ainda, podemos nomear constantes que representam um **conjunto enumerado**.

- Exemplo:

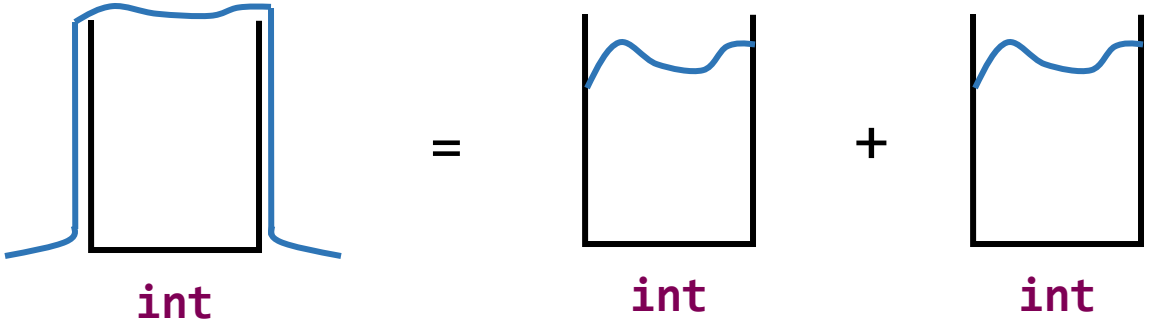
```
enum
{
    MOTOR_OFF          = 0,
    MOTOR_MIN_SPEED    = 1,
    MOTOR_MEDIUM_SPEED = 2,
    MOTOR_MAX_SPEED    = 3
}
```

Overflow aritmético

Overflow aritmético

- O *overflow* (sobrecarga) aritmético ocorre quando o resultado de uma operação aritmética é um número maior do que a variável pode acomodar.

- Exemplo:



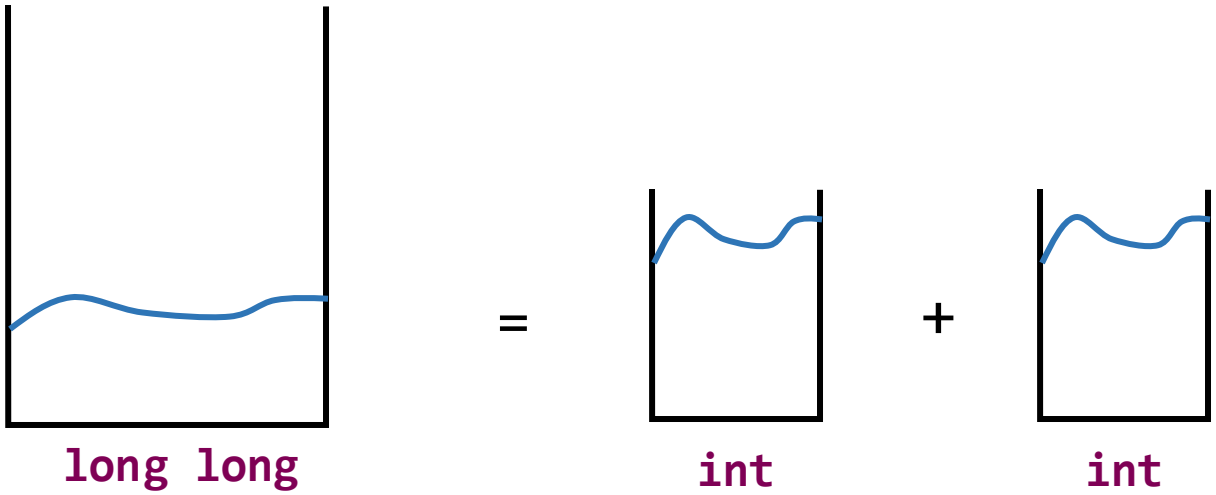
☞ Faixa de valores em um int:
-2,147,483,648 à 2,147,483,647

```
int x = 2 000 000 000 + 1 000 000 000;
```

Overflow aritmético

- Solução ao *overflow* aritmético: acomodar resultado em uma variável maior.

- Exemplo:



Faixa de valores em um int:
-2,147,483,648 à 2,147,483,647

Faixa de valores em um long long:
$-2^{64}/2$ à $2^{64}/2 - 1$

```
long long x = 2 000 000 000 + 1 000 000 000;
```

Expressões aritméticas

Expressões aritméticas

- O C++ oferece várias operações aritméticas que você pode usar para realizar cálculos em expressões:
  - Soma (+);
  - Subtração (-);
  - Multiplicação (*);
  - Divisão (/);
  - Módulo (%).

Expressões aritméticas

- Exemplos de operações de soma:

```
constexpr int b = 3
int a = 2;
char c = 4;
int soma = a + b; // soma = 5
soma += c; // soma = soma + c = 9

float z = 3.14, w = 14.007;
float somaF = z + w; // soma = 17.147
float y = -7;
somaF = somaF + y; // soma = 10.147
```

Expressões aritméticas

- Exemplos de operações de subtração:

```
#define A 2
#define B 3
...
int diff = A - B; // diff = -1

char z = 200, w = 33;
char diffC = z - w; // diffC = 167

diff -= diffC; // diff = -168
```

Expressões aritméticas

- O C++ também possui operações de **incremento** e **decremento**.

- Incremento: somar a variável com 1.

```
int a = 3;
a++; // a = a + 1 = 4
```

- Decremento: subtrair a variável com 1.

```
int a = 3;
a--; // a = a - 1 = 2
```

Expressões aritméticas

- Exemplos de operações de multiplicação:

```
unsigned short a = 1000U, b = 3U;
unsigned short mul = a * b; // mul = 3000U

float z = 10000, w = 9887.733;
float mulF = z * w; // mulF = 98877.330
```

Expressões aritméticas

- Exemplos de operações de divisão:

```
int a = 10, b = 5, c = 3;
int div = a / b; // div = 2
int div = b / c; // div = 1
int div = b / a; // div = 0
int div = b / 0; // será lançado erro!

float z = 15, w = 4;
float divF = z / w; // divF = 3.75
float divF /= w; // divF = 0.9375
```

Expressões aritméticas

- Exemplos de operações de módulo:

```
int a = 10, b = 3;
int resto = a % b; // resto = 1

int resto %= 1; // resto = 0

int resto = a % 5; // resto = 0
```

Expressões aritméticas

- Quando você realiza uma operação aritmética entre um número inteiro e um número de ponto flutuante, o C++ automaticamente converte o número inteiro para o tipo de ponto flutuante e, em seguida, realiza a operação.
- Isso ajuda a evitar perda de precisão e facilita o tratamento de diferentes tipos numéricos.

```
int a = 2;
float b = 3.14;

cout << a + b;
```

2.0 + 3.14 = 5.14

Expressões aritméticas

- Se o resultado de uma operação entre um número em ponto flutuante e um inteiro for atribuído a um inteiro, o resultado é truncado:

```
int a = 2;
float b = 3.14;

a = a + b; // div = 5
```

2.0 + 3.14 = 5.14

Expressões aritméticas

- O resultado abaixo segue o esperado no seu computador?

```
int x = 3.3 * 3 + 6.1;

cout << x << endl;
```

Expressões relacionais

Expressões relacionais

- Operadores relacionais em C++ retornam valores booleanos (true ou false)

- O C++ oferece várias operações relacionais que você pode usar para realizar comparação entre valores:
  - Igual a (==);
  - Diferente que (!=);
  - Menor que (<);
  - Maior que (>) ;
  - Menor ou igual a (<=);
  - Maior ou igual a (>=);

Expressões relacionais

- Exemplos de operações de “igual a” (==):

```
int a = 2, b = 3;

bool isEqual = a == b; // isEqual = false

a++; // a = a + 1 = 3

isEqual = a == b; // isEqual = true
```

Expressões relacionais

- Exemplos de operações de “diferente que” (!=):

```
int a = 2, b = 3;

bool isDiff = a != b; // isDiff = true

a++; // a = a + 1 = 3

isDiff = a != b; // isDiff = false
```

Expressões relacionais

- Exemplos de operações de “menor que” (<):

```
int a = 2, b = 3;

bool isLess = a < b; // isLess = true

a++; // a = a + 1 = 3

isLess = a < b; // isLess = false
```

Expressões relacionais

- Exemplos de operações de “maior que” (>):

```
int a = 2, b = 3;

bool isGreater = a > b; // isGreater = false

a++; // a = a + 1 = 3

isGreater = a > b; // isGreater = false

a++; // a = a + 1 = 4

isGreater = a > b; // isGreater = true
```

Expressões relacionais

- Exemplos de operações de “menor ou igual que” (<=):

```
int a = 2, b = 3;

bool isLessOrEq = a <= b; // isLessOrEq = true

a++; // a = a + 1 = 3

isLessOrEq = a <= b; // isLessOrEq = true

a++; // a = a + 1 = 4

isLessOrEq = a <= b; // isLessOrEq = false
```

Expressões relacionais

- Exemplos de operações de “maior ou igual que” (>=):

```
int a = 2, b = 3;

bool isGreatOrEq = a >= b; // isGreatOrEq = false

a++; // a = a + 1 = 3

isGreatOrEq = a >= b; // isGreatOrEq = true

a++; // a = a + 1 = 4

isGreatOrEq = a >= b; // isGreatOrEq = true
```

Expressões lógicas

Expressões lógicas

- Operadores lógicos em C++ retornam valores booleanos (true ou false)

- O C++ oferece várias operações lógicas que você pode usar para realizar cálculos em expressões:
  - E (AND) lógico (&&);
  - OU (OR) lógico (||);
  - NÃO (NOT) lógico (!).

Expressões lógicas

- **E Lógico (&&)**: Retorna true se ambas as expressões à esquerda e à direita forem true.

```
bool a = true, b = false;

bool resultado = a && b; // resultado = false
```

Expressões lógicas

- **OU Lógico (||)**: Retorna true se pelo menos uma das expressões for true.

```
bool a = true, b = false;

bool resultado = a || b; // resultado = true
```

Expressões lógicas

- **NÃO Lógico (!)**: Retorna true se a expressão for false, e false se a expressão for true.

```
bool a = true;

bool resultado = !a; // resultado = false

bool b = false;

b = !b;

resultado = a && b; // resultado = true
```

Expressões compostas

Expressões compostas

- Expressões compostas são aquelas que combinam múltiplas subexpressões.

- As expressões compostas podem ser divididas em duas categorias principais:

  - **Expressões Aritméticas Compostas**
  - **Expressões Lógicas Compostas**

Expressões compostas

- Expressões aritméticas compostas envolvem operações aritméticas para calcular valores resultantes.

- Elas podem incluir operadores aritméticos como adição, subtração, multiplicação, divisão e módulo, bem como **parênteses** para controlar a ordem de avaliação.

Expressões compostas

- Exemplos de expressões aritméticas compostas:

```
constexpr int b = 13
int a = 7;
int c;

c = (a + b)/2; // c = 10
c = a + b / 2; // c = 13
a = 5*(a + c + 2); // c = 110
```

Expressões compostas

- Expressões lógicas combinam subexpressões usando operadores lógicos, como && (E lógico) e || (OU lógico), juntamente com operadores relacionais para avaliar condições mais complexas.

- Ainda, podem haver instruções aritméticas dentro de expressões lógicas, cujo resultado é usado para avaliar o valor lógico.

Expressões compostas

- Exemplos de expressões lógicas compostas:

```
int idade = 23;
bool temCarteiraFisica = true;
bool temCarteiraDigital = false;

// condicao = false
bool ehMenor18 = ( ( idade + 1980 ) < 1998 );

// condicao = true
bool PodeDirigir = ( !ehMenor18 &&
                     ( temCarteiraFisica ||
                       temCarteiraDigital )
                   );
```

Expressões, variáveis e constantes

Casting

Casting

- ***Casting*** é um recurso que converte um determinado valor/variável para outro tipo daquele original.

- Os métodos de conversão podem ser classificados em dois tipos: **seguros** e **não seguros**

Casting

- O comando **`static_cast`** realiza uma **conversão segura** de um valor em outro tipo, em **tempo de compilação**.

- Ex. 1:

```
int x;
char c = 2;

x = static_cast<int>(c);
```

☞ O valor de 8 bits dentro do char tem o mesmo significado do que um valor correspondente em 32 bits do int

Casting

- O comando **`static_cast`** realiza uma **conversão segura** de um valor em outro tipo, em **tempo de compilação**.

- Ex. 2:

```
int x = 3000;
char c;

c = static_cast<char>(x);
```

- O valor de 32 bits dentro do int tem o mesmo significado do que um valor correspondente em 8 bits do char.
- Porém, caso o valor esteja fora da faixa -128 a 127, haverá truncamento do valor

Casting

- O comando **`static_cast`** realiza uma **conversão segura** de um valor em outro tipo, em **tempo de compilação**.

- Ex. 3:

```
int x = 3000;
float f;

f = static_cast<float>(x);
```

- O valor de 32 bits do int será convertido em um valor de 32 bits no float que tem o mesmo significado que o valor de 32 bits no int.

Casting

- O comando **`dynamic_cast`** é semelhante ao `static_cast`, porém a conversão é feita em **tempo de execução**.

- Caso a conversão não seja permitida, o valor retornado será **`nullptr`**.

- Utilizada para tratar **polimorfismo** em **programação orientada a objetos**.

Casting

- O comando **`reinterpret_cast`** realiza uma **conversão NÃO segura** de um valor em outro tipo, em **tempo de compilação**.

- Ele é usado de forma efetiva em **ponteiros** e **referências**, e dessa forma será visto mais adiante com detalhes.

Casting

- O comando **`reinterpret_cast`** realiza uma **conversão NÃO segura** de um valor em outro tipo, em **tempo de compilação**.

- Ex. 1:

```
int x = 3000;
float f;

f = reinterpret_cast<float>(x);
```

☞ Não vai compilar, pois não é possível reinterpretar 3000 em inteiro num valor diferente de 3000 em float.

Casting

- O comando **`reinterpret_cast`** realiza uma **conversão NÃO segura** de um valor em outro tipo, em **tempo de compilação**.

- Ex. 2:

```
int x = 3000;
double y =
reinterpret_cast<double&>(x);
```

Note que o "&", o operador de **referência**, muda o jogo aqui.
Quer dizer basicamente: reinterprete os bits guardados em x, guardando em y.